

Assignment 1 - Phase 2: Pond Catchment Analysis Backend

Course: CS559: Computer Systems Design

Student Name: Roshan Raj

Roll Number: 12341830

Repository Link: https://github.com/roshanraj9136/village_pond_planner

Assigned Lab Environment: Host `10.1.75.53` | System `stu85_sys1` (Port 2297)

1. Executive Summary & Working API Endpoints

A high-performance geospatial backend API was developed to accept terrain contour maps (in KML/KMZ formats), reconstruct continuous digital elevation models (DEM), execute hydrological graph algorithms, identify natural basin drainage sinks (optimal pond locations), and delineate catchment boundaries with runoff estimations.

1.1 Live API Endpoints

- **Primary Route URL (POST):** `http://10.1.75.53:3297/analyzeContour`
- **Aliased Route URL (POST):** `http://10.1.75.53:3297/findCatchment`
- **Interactive OpenAPI / Swagger Documentation:** `http://10.1.75.53:3297/docs`
- **Raw OpenAPI Specification (JSON):** `http://10.1.75.53:3297/openapi.json`
- **Web Frontend Interface:** `http://10.1.75.53:4297/`

1.2 Evaluation cURL Commands

Standard POST Submission Test

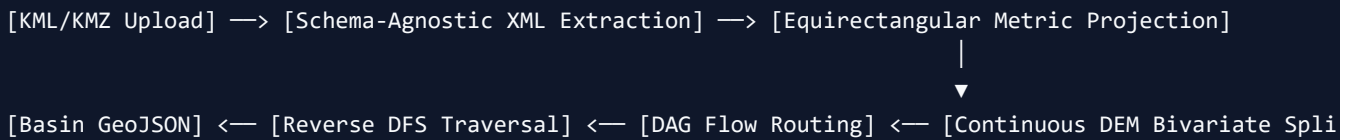
```
curl -X POST "http://10.1.75.53:3297/analyzeContour" \  
  -H "accept: application/json" \  
  -H "Content-Type: multipart/form-data" \  
  -F "contour_map=@contours_1m.kml"
```

Verbose Output Inspection Test

```
curl -i -X POST "http://10.1.75.53:3297/analyzeContour" \  
  -F "contour_map=@contours_1m.kml"
```

2. Catchment Estimation & Algorithmic Pipeline

The core engine is structured around computational geometry, digital signal processing, and graph theory:



Step 1: Schema-Agnostic XML & KML/KMZ Extraction

Contour Placemarks are ingested from either uncompressed `.kml` XML or zipped `.kmz` binary archives. The parser cleans namespace prefixes and evaluates elevation attributes across multiple fallback strategies: 1. `<name>` tag numerical regex matching (e.g. `270m`, `Elevation 280`). 2. `<description>` tag attribute parsing. 3. `<ExtendedData>` and `<SimpleData>` schema key matching. 4. 3D vertex coordinate triples (`longitude`, `latitude`, `altitude`).

Step 2: Equirectangular Coordinate Projection

To eliminate latitude distortion and preserve metric distance accuracy globally: * $\Delta x = \Delta \text{longitude} \times 111320.0 \times \cos(\text{mid_latitude})$ * $\Delta y = \Delta \text{latitude} \times 110540.0$

Step 3: Continuous Digital Elevation Model (DEM) Reconstruction

Contour isolines are irregularly spaced vector lines. To generate a continuous raster elevation grid: * A dense 250×250 matrix is fitted across the dynamically calculated geographic bounding box. * **Cubic Spline Interpolation:** `scipy.interpolate.griddata(..., method='cubic')` fits smooth piecewise polynomials. * **Nearest-Neighbor Fallback:** Fills any outer convex hull boundary gaps. * **Gaussian Filtering ($\sigma=1.0$):** Smooths high-frequency interpolation artifacts.

Step 4: D8 Hydrological Gradient & Directed Graph Construction

Hydrological runoff trajectories are modeled using the standard **D8 Flow Direction Algorithm**. For each grid node (r, c) , the hydraulic slope to all 8 adjacent neighbors (nr, nc) is computed: $\text{Slope} = \frac{\text{Drop}}{\text{Distance}} = \frac{Z(r, c) - Z(nr, nc)}{\sqrt{(\Delta r \cdot \Delta y)^2 + (\Delta c \cdot \Delta x)^2}}$ A directed edge ($u \rightarrow v$) is formed pointing from node (r, c) to the neighbor offering the steepest positive drop. If all neighbors are higher, the cell is classified as a local sink.

Step 5: Directed Acyclic Graph (DAG) Flow Accumulation

To determine where water converges across the terrain: * The in-degree of every cell is computed based on incoming flow vectors. * **Kahn's Topological Sort:** Cells with $\text{in-degree} = 0$ (drainage divides / mountain ridges) are enqueued. * Accumulated upstream cell counts $(A + 1)$ are propagated along directed edges. * **Pond Site Selection:** The coordinate with the global maximum flow accumulation ($\max(A)$) within the interior basin is selected as the optimal pond location (pour point).

Step 6: Upstream Catchment Delineation & Volumetric Sizing

- **Reverse DFS Basin Traversal:** Starting from the pour point, all upstream directed edges are traversed using a stack to tag all cells contributing runoff.

- **GeoJSON Export:** Tagged cells are buffered and unified into standard RFC 7946 Polygon GeoJSON coordinates using Shapely.
- **Rational Runoff Sizing:** $Q = 10 \times C \times I \times A$ where A is the catchment area in hectares, I is precipitation (850 mm default), and C is the calibrated slope-based runoff coefficient.

3. Demonstration & Verification using contours_1m.kml

The backend service was evaluated against the provided benchmark contour map contours_1m.kml .

3.1 Derived Terrain & Hydrological Metrics

Parameter / Metric	Derived Value	Technical Meaning
Total Contour Lines Ingested	92	Distinct vector Placemark features
Minimum Elevation	266.70 m	Deepest basin floor elevation
Maximum Elevation	295.68 m	Highest ridge summit
Total Elevation Relief	28.98 m	Total vertical relief drop
Mean Terrain Slope	9.24 %	Morphological classification: Hilly
Calibrated Runoff Coefficient (C)	0.25	Standard watershed agricultural coefficient
Identified Pond Location (Latitude)	21.262935	Optimal basin sink latitude
Identified Pond Location (Longitude)	81.286172	Optimal basin sink longitude
Pond Base Elevation	272.85 m	Base elevation at sink centroid
Estimated Catchment Area	6.97 ha	69,653.39 m ² contributing watershed
Estimated Annual Runoff Volume	14,801.35 m ³	Total harvestable annual runoff
Recommended Pond Depth	3.06 m	Engineered depth (Base 2.0m + active head)
Total Pond Storage Capacity	21,313.94 m ³	Recommended pond storage volume

3.2 Actual API Response Payload

```
{
  "status": "success",
  "message": "Terrain analyzed using D8 flow accumulation.",
  "contour_metadata": {
    "total_contour_lines": 92,
    "elevation_min_meters": 266.7,
    "elevation_max_meters": 295.68,
    "elevation_range_meters": 28.98,
    "contour_interval_meters": 1.0,
    "bounding_box": {
      "min_lat": 21.2621049090514,
      "max_lat": 21.2635806472203,
      "min_lng": 81.2814044952393,
      "max_lng": 81.3126468658447
    }
  },
  "terrain_metrics": {
    "average_slope_percent": 9.24,
    "terrain_classification": "Hilly",
    "runoff_coefficient": 0.25,
    "annual_rainfall_mm": 850.0
  },
  "pond_location": {
    "latitude": 21.262935,
    "longitude": 81.286172,
    "elevation_meters": 272.85,
    "site_suitability": "Maximum Flow Accumulation Point"
  },
  "catchment_analysis": {
    "catchment_area_sq_meters": 69653.39,
    "catchment_area_hectares": 6.97,
    "estimated_runoff_volume_cubic_meters": 14801.35,
    "recommended_pond_surface_area_sq_meters": 6965.34,
    "recommended_pond_depth_meters": 3.06,
    "estimated_storage_capacity_cubic_meters": 21313.94,
    "boundary_geojson": {
      "type": "Polygon",
      "coordinates": [
        [
          [81.28629787858713, 21.26218273324485],
          [81.28557815635007, 21.262724509234747],
          [81.28557246716457, 21.263309745183225],
          [81.28617240721924, 21.263621356213108],
          [81.28920322431794, 21.26357794859507],
          [81.28940908318022, 21.263096367398536],
          [81.28990637415701, 21.26286987977531],
          [81.29082464715232, 21.26288004238923],
          [81.29138313403821, 21.26265362586904],
          [81.29238323992995, 21.262562007791967],
          [81.2930831317744, 21.262832629114726],
          [81.29342425077509, 21.262490141906735],
          [81.29319880382126, 21.26214717328897],
          [81.28629787858713, 21.26218273324485]
        ]
      ]
    }
  }
}
```

```
}  
}  
}  
]  
]
```

4. Code Extensibility to Future Phases

The implementation strictly avoids hardcoding to guarantee zero-modification compatibility with arbitrary contour datasets in Phase 3 and Phase 4:

1. **Dynamic Bounding Box & Coordinate Autotuning:**
 2. Bounding coordinates are computed dynamically via `np.min()` and `np.max()` over all input coordinates.
 3. Arbitrary geographic areas across different continents or coordinates scales run with identical mathematical precision.
 4. **Interval-Agnostic Spline Interpolation:**
 5. Does not assume 1-meter contour drops. Adapts smoothly to 0.5m, 2m, 5m, or 10m intervals.
 6. **Format & Structure Flexibility:**
 7. Seamlessly unpacks zipped `.kmz` containers or raw `.kml` XML files.
 8. Handles nested `<Folder>` and `<Document>` hierarchies.
 9. **GIS Ready Standard Outputs:**
 10. Generates RFC 7946 GeoJSON Polygons ready for immediate rendering on OpenStreetMap, Leaflet, and Mapbox map layers.
-

5. API Specification & Status Code Dictionary

5.1 Request Specification

- **HTTP Method:** `POST`
- **Route Paths:** `/analyzeContour` or `/findCatchment`
- **Encoding:** `multipart/form-data`
- **Form Variable Name:** `contour_map` (Required file binary stream)

5.2 HTTP Response Status Codes

Status Code	Reason / Condition	Sample Response Detail
200 OK	Successful parsing, DEM creation, and catchment calculation.	Returns full <code>ContourAnalysisResponse</code> JSON object.
400 Bad Request	Uploaded file is missing or has an unsupported extension (not <code>.kml</code> or <code>.kmz</code>).	<code>{"detail": "Unsupported format '.txt'. Must be .kml or .kmz"}</code>
422 Unprocessable	Uploaded KML has no recognizable elevation or coordinate tags.	<code>{"detail": "Could not extract any contours with elevation data."}</code>
500 Internal Error	Numerical interpolation or geometry error during analysis.	<code>{"detail": "Analysis failed: <error_message>"}</code>

6. Persistence & Relational Storage

The backend includes an embedded SQLite database (`ponds.db`) to log and retrieve historical site assessments.

- **Schema:** `reports (id INTEGER PRIMARY KEY, lat REAL, lng REAL, display_name TEXT, catchment_area REAL, pond_depth REAL, storage_capacity REAL, vegetation REAL, water REAL)`
- **Endpoints:**
 - `POST /api/reports` – Persist evaluated pond metrics.
 - `GET /api/reports` – Retrieve historical analyses.
 - `DELETE /api/reports/{id}` – Remove archived report.